

A GPU-Accelerated High-Performance Design for CRYSTALS-Dilithium Digital Signature

Hien Nguyen, Bertrand Cambou, and Tuy Tan Nguyen*

School of Informatics, Computing, and Cyber Systems, Northern Arizona University, USA

{tn598, bertrand.cambou, tuy.nguyen}@nau.edu

Abstract—Our approach leverages the massive parallelism and computational power of a modern Graphics Processing Unit (GPU) to accelerate CRYSTALS-Dilithium’s key operations. While post-quantum cryptographic algorithms like Dilithium offer robust security, they face computational challenges in high-throughput scenarios. Dilithium’s performance bottlenecks, particularly in polynomial arithmetic and random sampling, have limited its practical deployment. To address these challenges, we leverage GPU acceleration as a promising solution to enhance efficiency. Our approach develops novel GPU-centric algorithms and data structures tailored to Dilithium’s requirements. We focus on optimizing Number Theoretic Transforms (NTT), polynomial arithmetic, and random sampling, with an emphasis on batch processing and efficient memory management to maximize throughput. Experimental results demonstrate significant performance improvements over existing implementations. Our GPU-accelerated version consistently outperforms reference implementations, achieving speed-ups ranging from 53.7% to 59.9% for signature generation and 20.0% to 63.8% for verification across various security levels. These results highlight the potential of our approach in enabling efficient post-quantum cryptography for high-performance applications.

Index Terms—CRYSTALS-Dilithium, digital signatures, post-quantum cryptography, NTT, GPU, parallel processing

I. INTRODUCTION

The emergence of quantum computing poses a significant threat to existing cryptographic systems; particularly those relying on the hardness of integer factorization and discrete logarithm problems [1], [2]. In response to this challenge the field of post-quantum cryptography has emerged, where cryptographic systems that can withstand attacks from both classical and quantum computers have been developed. Lattice-based cryptography, with its robust security properties and relatively efficient implementations, has emerged as a leading contender among various post-quantum schemes.

CRYSTALS-Dilithium (Dilithium) [3], a lattice-based digital signature scheme, has recently been selected by the National Institute of Standards and Technology (NIST) as a standard for post-quantum digital signatures. This selection underscores its importance in the future of secure communications. However, as is typical for post-quantum algorithms, Dilithium faces challenges in terms of computational efficiency and practical deployment; especially in high-throughput scenarios. The increasing demand for secure digital transactions, coupled with the computational intensity of post-quantum algorithms, necessitates innovative approaches to implementation. The massive parallel processing capabilities of the

Graphics Processing Unit (GPU) present a promising platform for accelerating cryptographic operations. GPU acceleration has the potential to significantly enhance the performance of Dilithium; rendering it more viable for real-world applications that require high-speed signature generation and verification.

Dilithium has gained significant traction across various platforms in recent years. While previous software implementations [4]–[6] made notable progress in optimizing Dilithium, they were often hindered by limitations such as suboptimal parallelism, memory management or platform-specific constraints. Our GPU-based implementation addresses these challenges by fully exploiting GPU parallelism; thus employing advanced memory management techniques and optimizing polynomial arithmetic for enhanced performance across diverse environments. This approach offers a compelling alternative to dedicated hardware or Field Programmable Gate Array (FPGA) implementations [7]–[9]; which can be constrained by scalability, flexibility, or resource utilization. By leveraging the scalability and adaptability of modern GPU architectures, our solution is well-suited for large-scale and high-performance applications such as blockchain systems and quantum-resistant communication protocols.

The remaining parts of the paper are structured as follows: Section II covers preliminary knowledge of Dilithium and computational challenges. Section III outlines our proposed developments and implementations. In Section IV, we describe the result of this proposal and compare it with related work. Lastly, the paper concludes in Section V.

II. BACKGROUND

A. Dilithium

Dilithium is a lattice-based digital signature scheme that bases its security on the difficulty level of problems in module lattices. It is designed to provide post-quantum security while maintaining efficiency in terms of key and signature sizes. The scheme consists of three main algorithms: key generation, signing and verification. Dilithium’s operations primarily involve polynomial arithmetic in a ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ where q is a prime modulus and n is a power of 2. Table I lists the parameters of Dilithium for three NIST security levels.

B. Computational Challenges

Dilithium offers strong security in post-quantum cryptography but faces notable computational challenges. A primary issue is the high cost of polynomial multiplication, which is

TABLE I
DILITHIUM PARAMETERS FOR THREE NIST SECURITY LEVELS [1]

Security level	n	q	(k, ℓ)	d	τ	γ_1	γ_2	η	β	ω	L_1	L_2
2	256	8380417	(4,4)	13	39	2^{17}	95232	2	78	80	256	512
3	256	8380417	(6,5)	13	49	2^{19}	261888	4	196	55	256	512
5	256	8380417	(8,7)	13	60	2^{19}	261888	2	120	75	256	512

essential to the scheme. While the number theoretic transforms (NTT) speed up this process, it remains resource-intensive; especially at higher security levels. Additional polynomial operations also contribute to this computational overhead which extends processing times [6].

Managing large matrices and vectors of polynomials in three main algorithms leads to substantial memory usage which can be problematic in constrained environments [5], [10]. The signing process may require multiple attempts due to rejection sampling which introduces unpredictability in signing times (particularly in real-time applications) [4]. In high-traffic server environments, the large size of keys and signatures may cause input/output bottlenecks thereby necessitating optimizations to handle this overhead [1], [2].

GPUs significantly accelerate cryptographic tasks; particularly polynomial multiplication, due to their parallel processing capabilities and higher memory bandwidth [1], [11]. They also feature specialized arithmetic units for modular operations which enhances performance for post-quantum algorithms [7], [8]. However effective GPU utilization presents challenges that include efficient memory management and data transfer overhead between the Central Processing Unit (CPU) and GPU. Poor management limits performance gains while optimizing data transfers is crucial for improving the efficiency of GPU-accelerated cryptographic operations [1], [4]. By addressing these challenges and leveraging the strengths of GPU architecture it is possible to create a high-performance implementation of Dilithium that significantly accelerates post-quantum digital signature operations.

III. GPU ACCELERATION OF DILITHIUM

Our GPU implementation of Dilithium utilizes the substantial parallel processing capabilities of modern GPUs. This section details the proposed parallelization strategy, key optimizations, and the implementation of core Dilithium operations.

A. Parallelization Strategy

GPUs are specialized hardware designed for parallel processing tasks. In our implementation, we leverage the power of GPUs to perform multiple signing and verification processes simultaneously which significantly enhances performance.

To optimize our implementation we employ a fine-grain technique that allows multiple threads to collaborate on a single task. This approach takes advantage of the vast number of threads available in a GPU which leads to a more efficient resource utilization and improved overall throughput. By distributing smaller sub-tasks among several threads we achieve

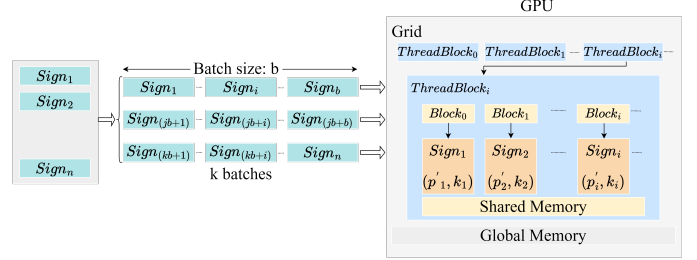


Fig. 1. Parallel signature operation design in a GPU environment.

faster execution times and better scalability. In our GPU implementation of Dilithium, we adopt a hybrid approach. Each GPU block comprises a group of threads that independently executes a single signing and verification process. Within each block, the threads work together to efficiently complete these tasks. This process is illustrated in Fig. 1 which shows our batch processing method for signing operations. Signing tasks are divided into batches of b tasks; with each block in the thread block processing one task. This approach allows for simultaneous execution of multiple signing tasks across different blocks in the GPU grid which effectively utilizes GPU resources and minimizes thread idle time.

By implementing this batch processing strategy we significantly enhance the performance of Dilithium in a GPU environment. This approach enables faster cryptographic operations and improved scalability (particularly for applications requiring high throughput). The batch size, (b) , may be tuned to optimize performance based on the specific GPU architecture and the requirements of the application.

B. Optimizations

Our implementation incorporates several optimizations across different aspects of the algorithm. We have developed a highly optimized implementation of the NTT that is specifically tailored for execution on a GPU. The NTT is a key component in many cryptographic algorithms; optimizing its performance leads to significant improvements in the overall system efficiency. We focus on two primary areas of optimization. First, we utilize butterfly operations which are essential for the NTT. These operations have been specifically optimized to align with the architecture of modern GPUs. Each butterfly operation transforms a pair of elements (u, v) using the twiddle factor, (w) , represented mathematically as $u = (u + wv) \bmod Q$ and $v = (u - wv) \bmod Q$ where Q is a large prime modulus and w is a primitive N -th root of unity. By optimizing

The second area of optimization involves the pre-computation and caching of twiddle factors in constant memory. This is crucial since the twiddle factors, (w) , are used repeatedly throughout the NTT stages. Fig. 2 illustrates the representation of the NTT stages execution in a GPU block. This flowchart effectively demonstrates the parallel execution of butterfly operations by multiple threads in a GPU block, highlighting their access to shared memory and the twiddle factors stored in constant memory. Algorithm 1 provides an in-depth overview of the kernel function’s structure. By pre-computing these factors and storing them in constant memory we eliminate the need for recalculating them during the transform; which not only reduces computational overhead but also enhances memory access speed. This results in a more efficient execution of the NTT thus allowing for faster processing of polynomial multiplications and other related operations.

The diagram illustrates the architecture of the proposed hardware accelerator. At the top, input registers provide y , A , s_1 , and s_2 to a series of butterfly operation (BO) blocks. Each BO block is represented by a yellow box with wavy arrows indicating data flow. These BO blocks are connected to a larger block containing a Texture/Constant Cache and a memory block for Twiddle factors. The cache and memory are connected via bidirectional arrows. A legend on the right defines the BO operation and the execution models: Single thread, Multi thread parallel, and Warp.

Butterfly operation (BO)

The BO operation is defined by the following equations:

$$u \rightarrow (u + vw) \bmod m$$

$$v \rightarrow (u - vw) \bmod m$$

The execution models are:

- Single thread
- Multi thread parallel
- Warp

Algorithm 1 NTT Kernel and Calling Function

operations, rendering it suitable for high-performance, post-quantum-secure applications.

Our approach to accelerating Dilithium utilized Python to leverage its extensive libraries and GPU acceleration capabilities. We employed CUDA Python bindings to interface with the NVIDIA GPU thereby harnessing the full potential of the RTX 3090 Ti for our cryptographic operations.

Table III compares the performance of various Dilithium implementations across three security levels. Lower Clock Cycle Counts (CCC) indicate better performance. Our method demonstrates consistently strong performance across all security levels and operations; excelling particularly in key gen and signature. In key gen our implementation achieves superior efficiency with CCC ranging from 4.1 thousand CCC (KCCC) to 4.7 KCCC across all security levels (significantly outperforming most related methods). For instance, at security level 5, our approach (4.5 KCCC) is dramatically faster than [11] (108 KCCC) and [8] (124.8 KCCC). For signature generation, our method maintains consistent efficiency with CCC between 24.1 KCCC and 26.5 KCCC across all security levels. This performance surpasses most competitors (especially at higher security levels). For example, at security level 3, our implementation (26.5 KCCC) outperforms [11] (49.5 KCCC) and [8] (230.2 KCCC). In verification, our implementation shows consistent performance across all security levels (14.5 KCCC

Security Level	Key Gen [μ s]		Signature [μ s]		Verification [μ s]	
	Our Results	[12]	Our Results	[12]	Our Results	[12]
2	24	-	82	178	44	55
3	39	-	115	287	47	90
5	69	-	170	371	53	144

TABLE III
PERFORMANCE COMPARISON OF DILITHIUM IMPLEMENTATIONS: CCC (IN THOUSANDS)

Security Level	2			3			5		
Work	Key Gen	Signature	Verification	Key Gen	Signature	Verification	Key Gen	Signature	Verification
Our Results	4.1	24.1	14.5	4.7	26.5	15	4.5	25.8	15.7
[11]	4.3	30.1	4.9	7.3	49.5	7.9	108	55.2	12.6
[9]	18.2	22.5	33.2	22.9	24.5	44.9	-	-	-
[8]	41.7	143.5	19.5	72.5	230.2	26.5	124.8	274.7	40.3
[13]	165	345	165	225	465	240	330	675	345

to 15.7 KCCC). While not always the fastest for verification our method offers balanced performance and outperforms most competitors at higher security levels. For instance, at security level 5, our implementation (15.7 KCCC) is more efficient than [8] (40.3 KCCC) and [13] (345 KCCC). Note that our method significantly outperforms [8] across all operations and security levels. We also show substantial improvements over [13] (especially in key generation and signature generation at higher security levels). The implementation by [11] shows competitive performance in verification; particularly at lower security levels. However, our implementation maintains more consistent performance across all operations and security levels. The result in [9] shows competitive performance at lower security levels; particularly in signature generation. These results indicate that our implementation is well-optimized; thereby offering a good balance of performance across various cryptographic operations and security levels. This consistent performance across all aspects makes our implementation particularly suitable for applications requiring high efficiency in post-quantum cryptography on GPU platforms.

V. CONCLUSION

We highlighted the potential of GPU acceleration to significantly enhance the performance of Dilithium digital signatures, a critical step towards efficient post-quantum cryptography. Our evaluation results demonstrate that our GPU-accelerated Dilithium implementation achieves a better performance compared to existing CPU-based methods. This improved performance could be beneficial in high-throughput applications such as IoT devices, cloud computing, and blockchain systems. Future efforts will focus on optimizing verification processes and assessing their application in real-world and high-throughput systems.

ACKNOWLEDGMENT

Research was sponsored by the Army Research Laboratory and was accomplished under Cooperative Agreement Number W911NF-23-2-0014. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the Army Research Laboratory or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation herein.

REFERENCES

- [1] S. Shen, H. Yang, W. Dai, H. Zhang, Z. Liu, and Y. Zhao, "High-throughput GPU implementation of dilithium post-quantum digital signature," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 11, pp. 1964–1976, Nov. 2024.
- [2] L. Beckwith, D. T. Nguyen, and K. Gaj, "High-Performance Hardware Implementation of CRYSTALS-Dilithium," in *2021 International Conference on Field-Programmable Technology (ICFPT)*, Auckland, New Zealand, 06–10 Dec. 2021, pp. 1–10.
- [3] NIST, "Module-lattice-based digital signature standard," Federal Information Processing Standards Publication – FIPS 204, Aug. 2024. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.204>
- [4] P. V. Ravi, S. S. Gupta, A. Chattopadhyay, and S. Bhasin, "Improving speed of Dilithium's signing procedure," in *Smart Card Research and Advanced Applications*, Prague, Czech Republic, 11–13 Nov. 2019, pp. 57–73.
- [5] D. O. C. Greconici, M. J. Kannwischer, and A. Sprenkels, "Compact Dilithium implementations on Cortex-M3 and Cortex-M4," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2021, no. 3, pp. 1–24, Dec. 2020.
- [6] H. Becker, V. Hwang, M. Kannwischer, B.-Y. Yang, and S.-Y. Yang, "Neon NTT: Faster Dilithium, Kyber, and Saber on Cortex-A72 and Apple M1," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2022, pp. 221–244, Nov. 2021.
- [7] A. P. C. Utsav Banerjee, Tenzin S. Ukyab, "Sapphire: A configurable crypto-processor for post-quantum lattice-based protocols," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, no. 4, pp. 17–61, Aug. 2019.
- [8] G. Land, P. Sasdrich, and T. Güneysu, "A Hard Crystal - Implementing dilithium on reconfigurable hardware," in *Smart Card Research and Advanced Applications: 20th International Conference, CARDIS 2021, Lübeck, Germany*, vol. 13173, no. 21, Lübeck, Germany, 11–12 Nov. 2021, pp. 210–230.
- [9] S. Ricci, L. Malina, P. Jedlicka, D. Smekal, J. Hajny, P. Cibik, and P. Dobias, "Implementing CRYSTALS-Dilithium signature scheme on FPGAs," in *Proceedings of the 16th International Conference on Availability, Reliability and Security*, no. 1, Vienna, Austria, 17–20 Aug. 2021, pp. 1–11.
- [10] C. Zhao, N. Zhang, H. Wang, B. Yang, W. Zhu, Z. Li, M. Zhu, S. Yin, S. Wei, and L. Liu, "A compact and high-performance hardware architecture for CRYSTALS-Dilithium," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2022, no. 1, pp. 270–295, Nov. 2021.
- [11] X. Li, J. Lu, D. Liu, A. Li, S. Yang, and T. Huang, "A high speed post-quantum crypto-processor for Crystals-Dilithium," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 1, pp. 435–439, Jan. 2024.
- [12] S. Kudo, Y. Nogami, S. Huda, and Y. Koda, "Optimizing CRYSTALS-Dilithium in Rust: Radix-4 NTT and assembly-level comparison with official C implementation," in *2024 International Conference on Consumer Electronics*, Taichung, Taiwan, 09–11 Jul. 2024, pp. 395–396.
- [13] G. Mao, D. Chen, G. Li, W. Dai, A. I. Sanka, C. K. Koç, and R. C. C. Cheung, "High-performance and configurable SW/HW co-design of post-quantum signature CRYSTALS-Dilithium," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 16, no. 44, pp. 1–28, Jun. 2023.